Budget-Constrained Two-Stage Stochastic Optimization for LLM Routing

Yale Tsai Branden Chen

May 2026

1 Introduction

We consider a hypothetical agentic AI company that provides a low-cost general-purpose AI assistant. Many users rely on LLMs to solve everyday problems, study, write code, and answer technical questions, but a monthly subscription price of around $20 can still be expensive. Our company aims to lower this barrier by offering a centralized routing platform at a lower subscription price, such as around $10 per month, while still maintaining strong response quality across math, coding, and knowledge-based questions.

The central operational problem is LLM routing. For every incoming prompt, the system must decide which model should handle the request. Routing every prompt to the strongest frontier model may produce high-quality responses, but it would be too expensive for a low-cost platform. Routing every prompt to a free or lightweight model would reduce cost, but response quality may suffer. Thus, the company must balance performance and cost by selecting a small but effective model pool and routing each prompt to an appropriate model.

We formulate this problem as a two-stage stochastic mixed-integer optimization model. In the first stage, the company selects a limited pool of models to deploy. In the second stage, prompts are assigned to selected models. The stochastic element comes from uncertainty in future prompt demand. Since the true distribution of future prompts is unknown, we approximate expected performance using sample-average approximation (SAA) over the benchmark prompts and study robustness by reweighting the empirical prompt distribution across domains.

We focus on two research questions:

RQ1: How large must the deployed model pool be to achieve strong routing performance under a fixed average budget? Do we observe diminishing marginal returns as pool size increases?

RQ2: How sensitive is your solution to changes in the prompt distribution? Is it robust under different ambiguity sets of response distributions?

2 Data and Exploratory Analysis

We use the provided RouterBench dataset, which contains cost and performance results for 33 models evaluated on 240 prompts. The prompts are evenly sampled from four benchmark domains: AIME, GPQA, LCB, and MMLU-Pro. Each row represents one prompt–model pair and includes the benchmark name, prompt identifier, model name, binary score, and cost.

Before modeling, we performed exploratory data analysis to understand the cost-performance structure of the dataset. The main observation is that model costs are highly skewed: most paid calls are relatively cheap, but a small number of frontier models, such as Gemini-2.5-Pro and GPT-5, are much

more expensive. At the same time, several open-source or low-cost models have zero or near-zero API cost but weaker average accuracy. This creates a clear trade-off: using only the strongest models is too expensive for our low-cost platform, while using only free models may reduce quality. AIME also appears to be the most difficult and most expensive benchmark on average, which matters for the robustness analysis. The full EDA plots, including benchmark difficulty, model ranking, cost distribution, cost-performance trade-off, prompt difficulty, and token usage, are reported separately in Appendix Figures A1–A9.

3 Optimization Model

3.1 Sets and Parameters

Let P be the set of prompts, M be the set of candidate models, and D be the set of benchmark domains. For each prompt–model pair (p, m), define:

- $s_{pm} \in \{0, 1\}$: performance score of model m on prompt p;
- $c_{pm} \geq 0$: cost of using model m on prompt p;
- $w_p \geq 0,\ \sum_{p \in \mathcal{P}} w_p = 1$: SAA prompt weight approximating prompt arrival probability;
- $K \in \mathbb{Z}_+$: maximum number of models allowed in the deployed pool;
- $B \geq 0$: maximum weighted-average cost per prompt;
- $Q_d \in [0, 1]$: minimum required average score on dataset $d \in \mathcal{D}$;
- $\lambda \geq 0$: penalty per unit of quality-constraint violation.
- $y_m \in \{0, 1\}$: Stage-1 variable; equals 1 if model m is selected into the pool;
- $x_{pm} \in \{0, 1\}$: Stage-2 variable; equals 1 if prompt p is routed to model m;
- $\xi_d \geq 0$: auxiliary variable capturing quality violation on dataset d.

We also define $d(p)$ as the benchmark domain of prompt p. For optional soft quality guarantees, let Q_d be the target average score for domain d, and let λ be the penalty for violating this target.

3.2 Decision Variables

The first-stage model selection variable is:

$$y_m = \begin{cases} 1, & \text{if model } m \text{ is selected into the deployed pool,} \\ 0, & \text{otherwise.} \end{cases}$$

The second-stage routing variable is:

$$x_{pm} = \begin{cases} 1, & \text{if prompt } p \text{ is routed to model } m, \\ 0, & \text{otherwise.} \end{cases}$$

For soft quality constraints, we also define $\xi_d \geq 0$, where ξ_d is the quality violation allowed for benchmark domain d.

3.3 Two-Stage Stochastic Mixed-Integer Program

Our main model maximizes expected routing performance under a fixed budget:

$$\max_{x,y,\xi} \quad \sum_{p\in P} w_p \sum_{m\in M} s_{pm} x_{pm} - \lambda \sum_{d\in D} \xi_d.$$

The model is subject to:

$$\sum_{m\in M} y_m \le K,$$

$$\sum_{m\in M} x_{pm} = 1, \quad \forall p \in P,$$

$$x_{pm} \le y_m, \quad \forall p \in P,\ m \in M,$$

$$\sum_{p\in P} w_p \sum_{m\in M} c_{pm} x_{pm} \le B,$$

and the optional soft domain-level quality guarantee:

$$\frac{1}{|P_d|} \sum_{p\in P_d} \sum_{m\in M} s_{pm} x_{pm} \ge Q_d - \xi_d, \quad \forall d \in D,$$

where $P_d = \{p \in P : d(p) = d\}$. In the main experiments, we set $Q_d = 0$, so the soft quality guarantee is inactive. We keep it in the formulation because a production system may later require minimum quality guarantees for specific domains.

This is a two-stage model because the company first chooses a deployed pool y_m, and then assigns prompts through x_{pm}. It is stochastic because the objective approximates expected performance over random future prompts using empirical weights w_p. By changing w_p, we test robustness under different demand scenarios.

4 Implementation

We implement the model in Python using Pyomo and solve the mixed-integer program with HiGHS. The main solver constructs dictionaries for scores and costs, builds SAA prompt weights, solves the two-stage routing model, and extracts selected models, assignments, weighted average score, and weighted average cost. The main experiments use a budget of $B = 0.01$. This budget is lower than the average cost of expensive frontier models but high enough to allow occasional use of stronger paid models when their performance gain justifies the cost.

There are total of 4 hyperparameter, here is the setup of out hyperparameter:

- $K \in \mathbb{Z}_+$: For the question, we set it iterating from integer 1 to 10 (inclusive)
- $B \ge 0$: set 0.01
- $Q_d \in [0, 1]$: Change based on question 2, there are more specific detail in section 5
- $\lambda \ge 0$: Set 10

5 Results

5.1 RQ1: Model Pool Size and Diminishing Returns

For the first research question, we fix the budget at $B = 0.01$ and solve the model for $K = 1, \ldots, 10$. The pool-size sweep is shown in Appendix Figure A10. The results show a clear diminishing-returns pattern. When K increases from 1 to 2, weighted average score increases substantially, from about 0.742 to about 0.904. Increasing from $K = 2$ to $K = 3$ also provides a meaningful gain, reaching about 0.950. However, after $K = 5$, performance nearly plateaus: the score at $K = 5$ is about 0.979, while the score from $K = 6$ to $K = 10$ remains around 0.983.

The cost curve in Appendix Figure A10 shows that the model uses almost the full budget for most values of K. This is expected because the objective maximizes performance and cost is imposed as a constraint. The cost is not strictly increasing in K; for example, the cost at $K = 6$ is slightly lower than at $K = 5$. This occurs because a larger model pool expands the feasible set and may allow the optimizer to find cheaper model-prompt matches while maintaining or improving performance.

From a business perspective, $K = 5$ is a practical recommendation. It achieves near-frontier performance under the budget while keeping the deployed model pool small enough for a low-cost AI platform. Although a larger pool does not necessarily increase average per-prompt API cost, it does increase integration, monitoring, and maintenance complexity.

5.2 RQ3: Robustness Under Prompt Distribution Shift

For the second research question, we evaluate sensitivity to prompt distribution shift. The original dataset is balanced across AIME, GPQA, LCB, and MMLU-Pro, but real users may not be balanced. A coding-focused product may receive more LCB prompts, while a tutoring product may receive more AIME prompts. We approximate this uncertainty by changing the SAA weights w_p using dataset multipliers.

We consider four scenarios: balanced, coding-focused with LCB multiplied by 4, math-focused with AIME multiplied by 4, and knowledge-focused with GPQA and MMLU-Pro receiving higher weights. Appendix Figure A11 shows that weighted average score remains high across all scenarios. The coding-focused scenario performs the best, while the math-focused scenario performs slightly worse. This is consistent with the EDA finding that AIME is the most difficult benchmark.

Appendix Figure A12 compares selected model pools across scenarios. Several models appear repeatedly, such as GLM-Z1-9B-0414, Llama-3.1-8B-UltraMedical, GLM-4.6, GPT-5, and Kimi-K2-0905. This suggests that the optimal pool is not completely unstable under distribution shift. However, the math-focused scenario selects NVIDIA-Nemotron-Nano-9B-v2 and Gemini-2.5-Flash instead of some models selected in the balanced and coding-focused scenarios, indicating that math demand changes the best model combination.

To further evaluate robustness, we also train pools specialized for a single domain and evaluate them across all domains. Appendix Figure A13 shows the cross-domain performance matrix. The diagonal entries represent in-distribution performance, while off-diagonal entries represent out-of-distribution performance. Appendix Figure A14 summarizes in-distribution and out-of-distribution averages. In-distribution performance is higher, but out-of-distribution performance remains relatively strong. Thus, our routing model is reasonably robust, although performance can decrease when the deployment distribution differs from the distribution used to select the pool.

6 Discussion and Recommendations

Our first finding is that a small model pool can achieve strong performance under a fixed budget. The score improves rapidly when increasing the pool size from one to five models, but additional models

beyond five or six provide little marginal benefit. For our low-cost AI assistant company, this means that deploying around five models is a practical balance between quality and operational complexity. A six-model pool may be reasonable if the company wants slightly higher quality, but a ten-model pool is not necessary because it gives almost no additional performance improvement.

Our second finding is that the routing policy is relatively robust to prompt distribution shifts. When the workload becomes coding-focused or knowledge-focused, the model still achieves high weighted average score under the same budget. The math-focused scenario is more challenging, which matches the EDA result that AIME is difficult and costly. For a company expecting many math users, the pool should be tuned more carefully toward math-capable models.

From a production perspective, the proposed solution is suitable for a low-cost centralized AI platform because it does not rely entirely on expensive frontier models. Instead, it selects a small pool of complementary models and uses the budget constraint to prevent excessive spending. This supports the business goal of offering a lower subscription price while still solving hard questions through selective use of stronger models.

There are several limitations. First, benchmark prompts may not fully represent real user prompts. Second, model outputs are treated as fixed scores, but real LLM outputs are stochastic. Third, the current formulation does not include latency, storage, provider contracts, or user satisfaction. With more time, we would extend the model to include latency constraints, deployment storage costs for local models, and cascading routing with a verifier that escalates hard prompts to stronger models only when needed.

7 Personal Takeaway

This project helped us understand that LLM routing is not only an AI problem, but also an operations research problem. The key challenge is not simply finding the best model, but deciding how to allocate requests across models under cost, quality, and deployment constraints. Through the project, we learned how to translate a business problem into a two-stage stochastic mixed-integer optimization model, implement it in Pyomo, and use experiments to support practical recommendations. The most interesting takeaway is that a small pool of carefully selected models can achieve almost the same performance as a much larger pool, which is valuable for building affordable AI products.

Appendix

All supporting figures and code are provided in the separate appendix document.

- Appendix A: Exploratory Data Analysis (Figures A1–A9)
- Appendix B: RQ1 Results (Figure A10)
- Appendix C: RQ2 Results (Figures A11–A14)
- Appendix D: Pyomo Implementation

A. Exploratory Data Analysis

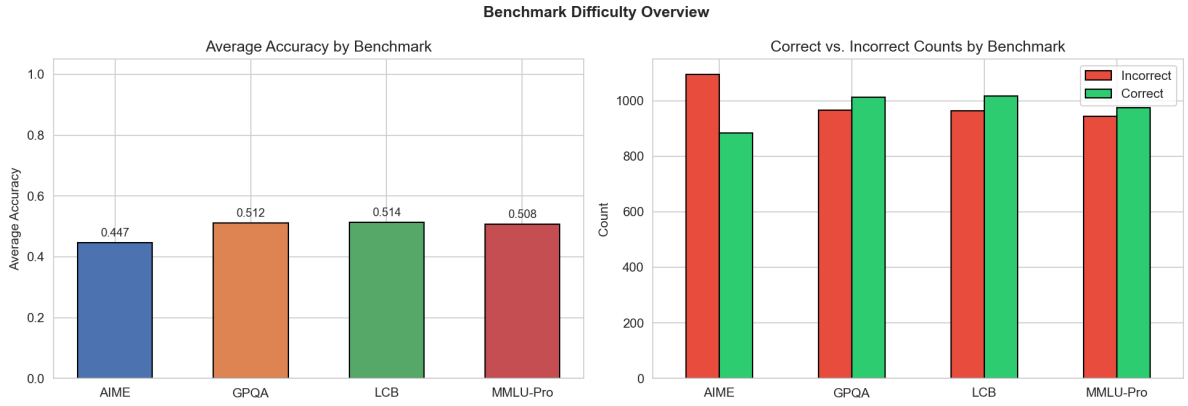

Figure A1: Benchmark difficulty overview. AIME has the lowest average accuracy, suggesting that math prompts are relatively harder.

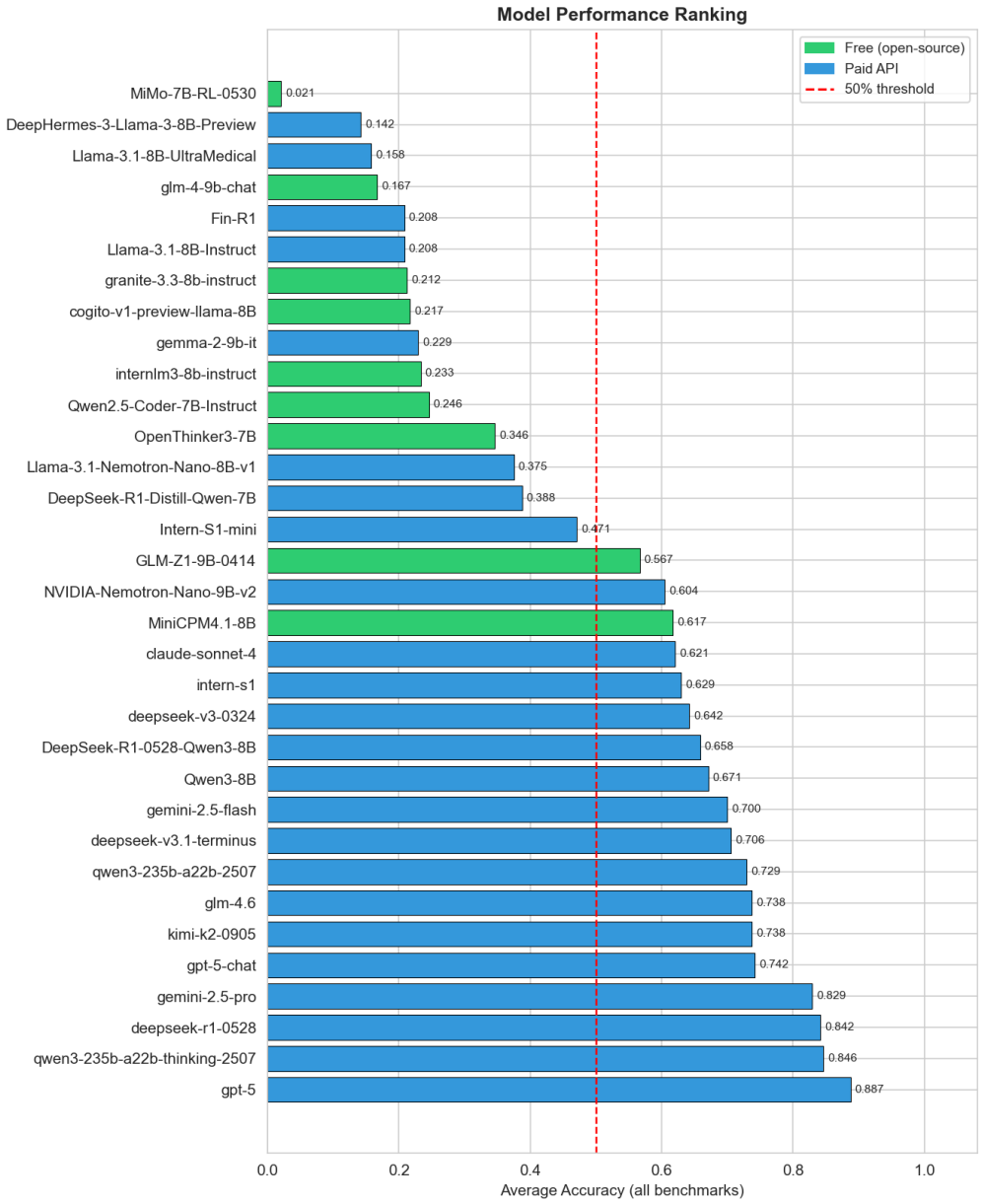

Figure A2: Model performance ranking by average accuracy. Frontier models perform strongly, but some free models remain valuable for cost-constrained routing.

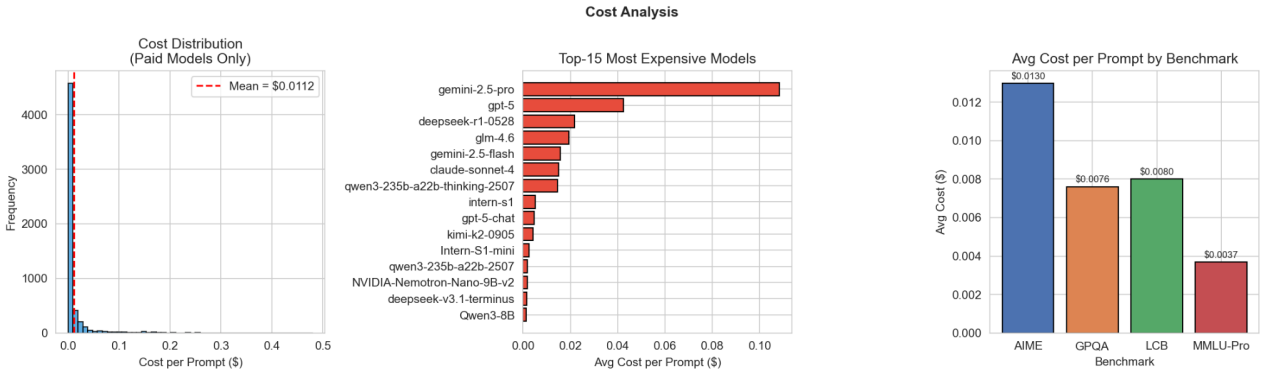

Figure A3: Cost analysis of the RouterBench data. Model costs are highly skewed, with a small number of frontier models being much more expensive than the rest. AIME also has the highest average cost per prompt.

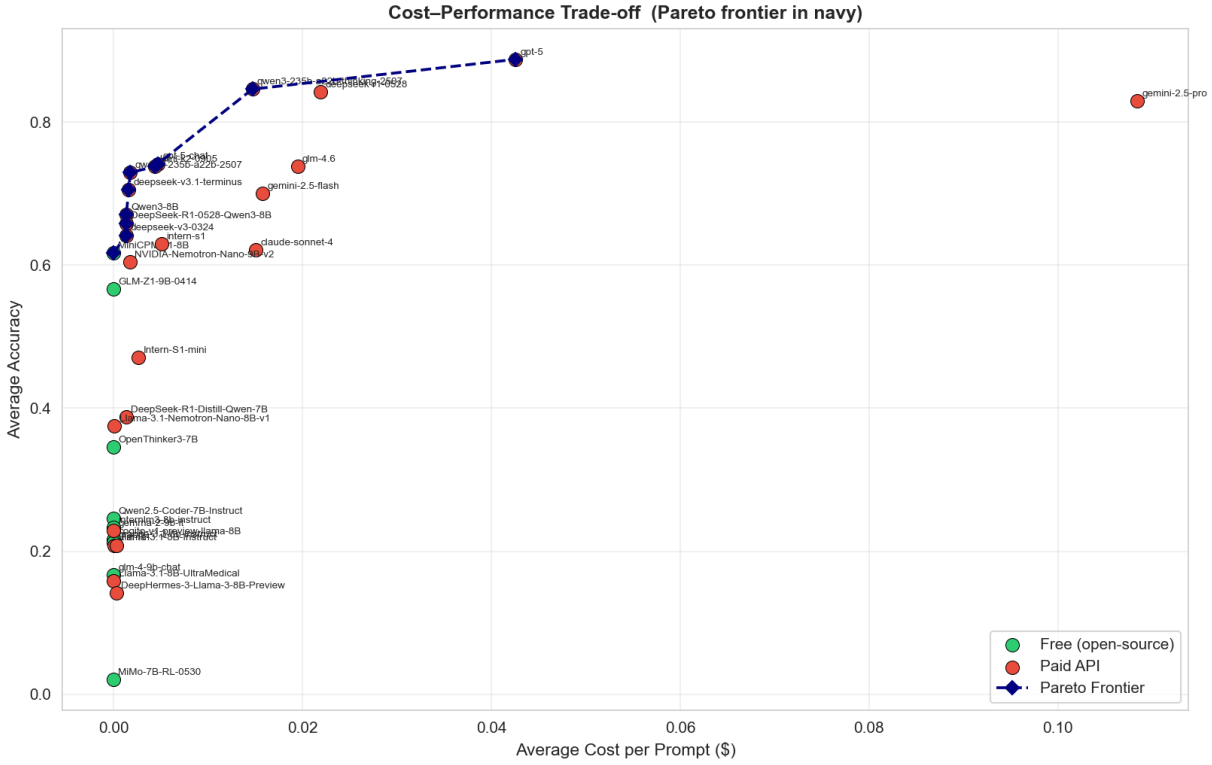

Figure A4: Cost–performance trade-off across models. The Pareto frontier motivates budget-constrained routing instead of using a single model for all prompts.

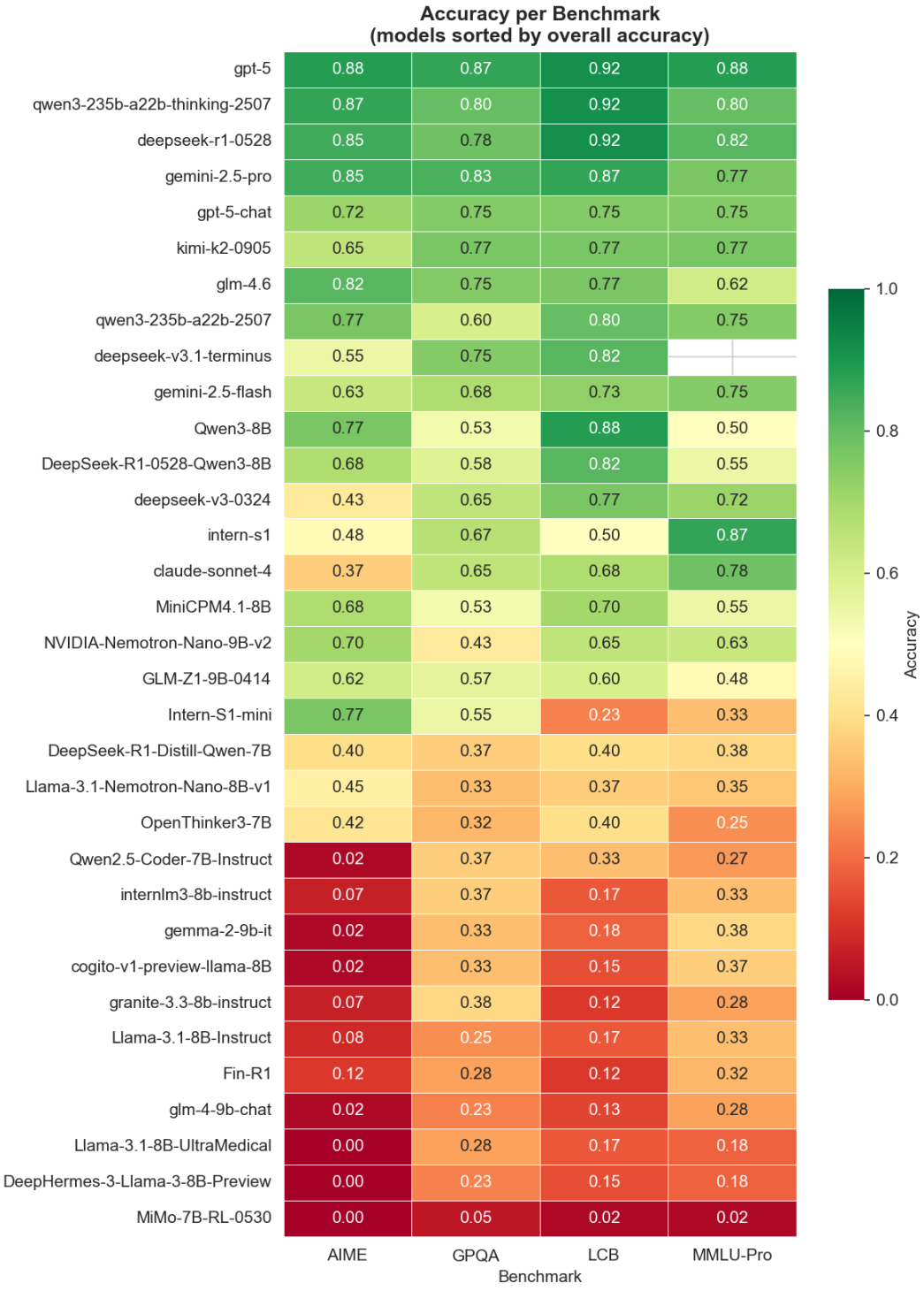

Figure A5: Accuracy by benchmark and model. Different models specialize in different benchmark domains.

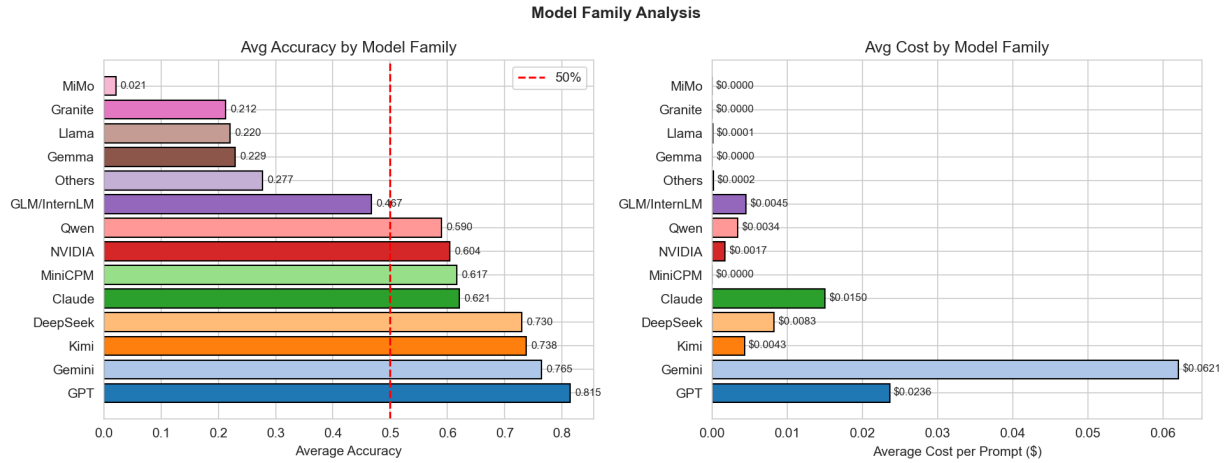

Figure A6: Model family analysis. Model families differ substantially in average accuracy and average cost.

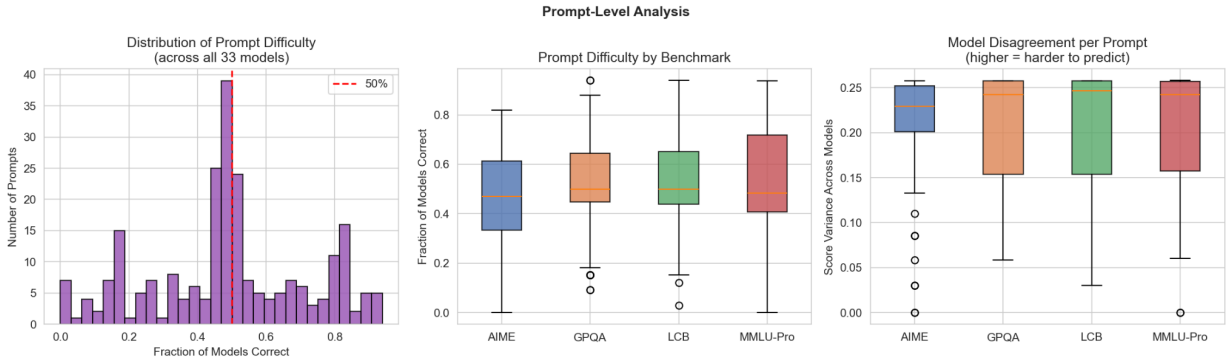

Figure A7: Prompt-level difficulty and model disagreement. This supports the need for prompt-specific routing decisions.

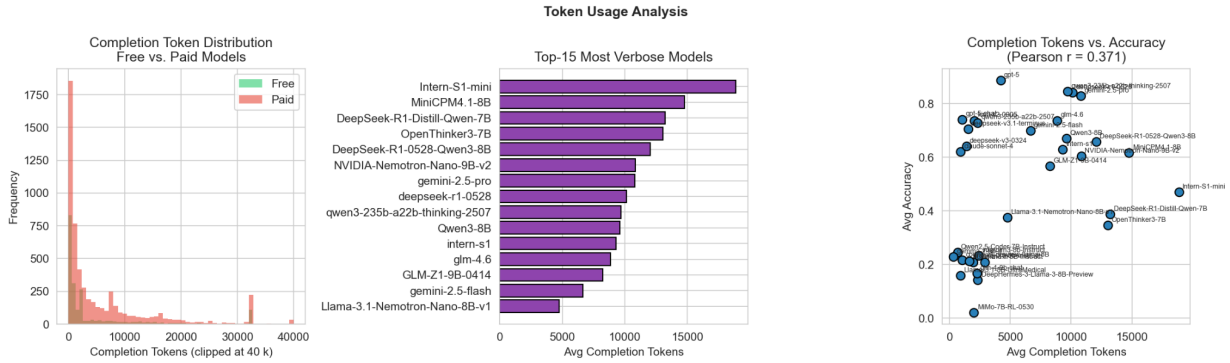

Figure A8: Token usage analysis across models. Completion length varies by model and is related to cost differences.

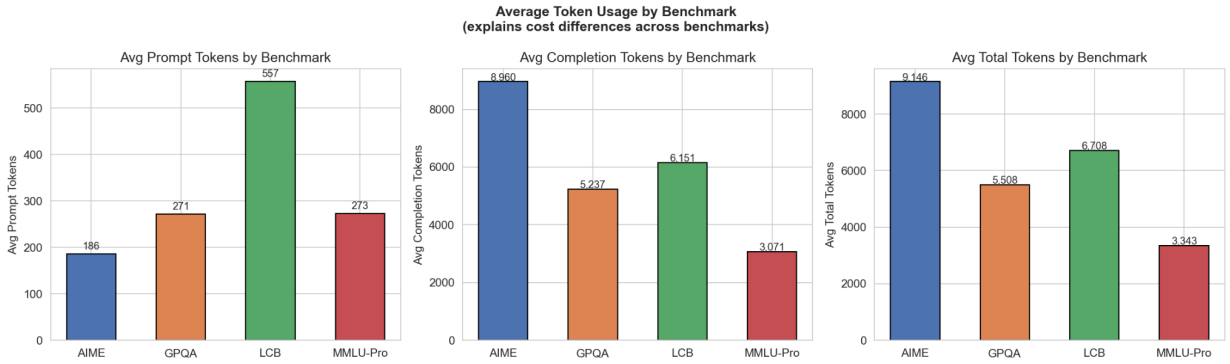

Figure A9: Average token usage by benchmark. Token usage helps explain why some benchmark domains are more expensive to serve.

B. RQ1: Pool Size and Diminishing Returns

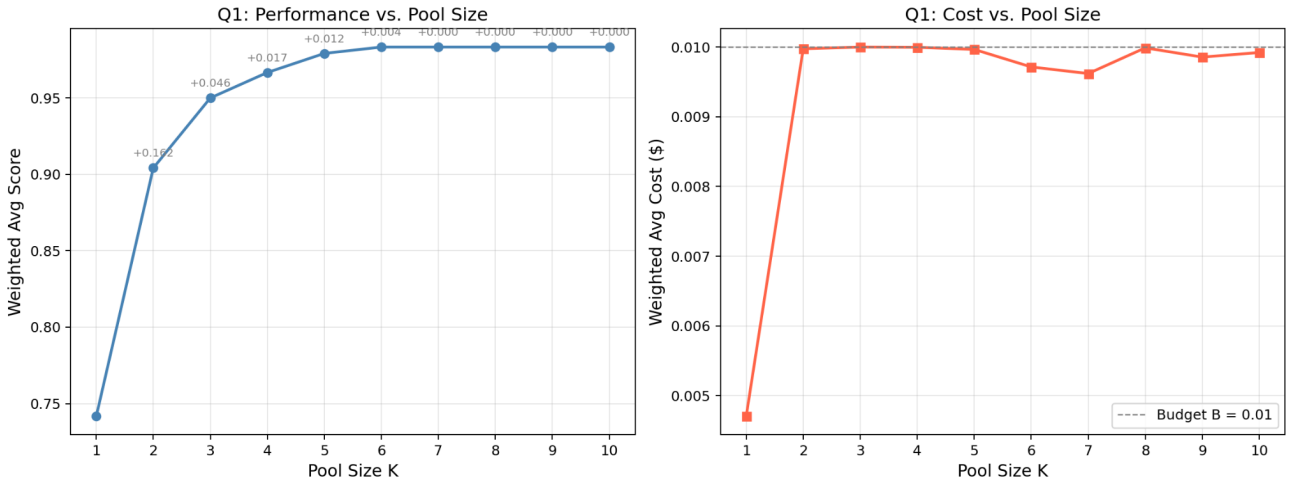

Figure A10: Pool size sweep under budget $B = 0.01$. Performance increases quickly from $K = 1$ to $K = 5$ and then plateaus, demonstrating diminishing marginal returns. The cost curve shows the model uses close to the full budget across most values of K.

C. RQ2: Robustness Under Prompt Distribution Shift

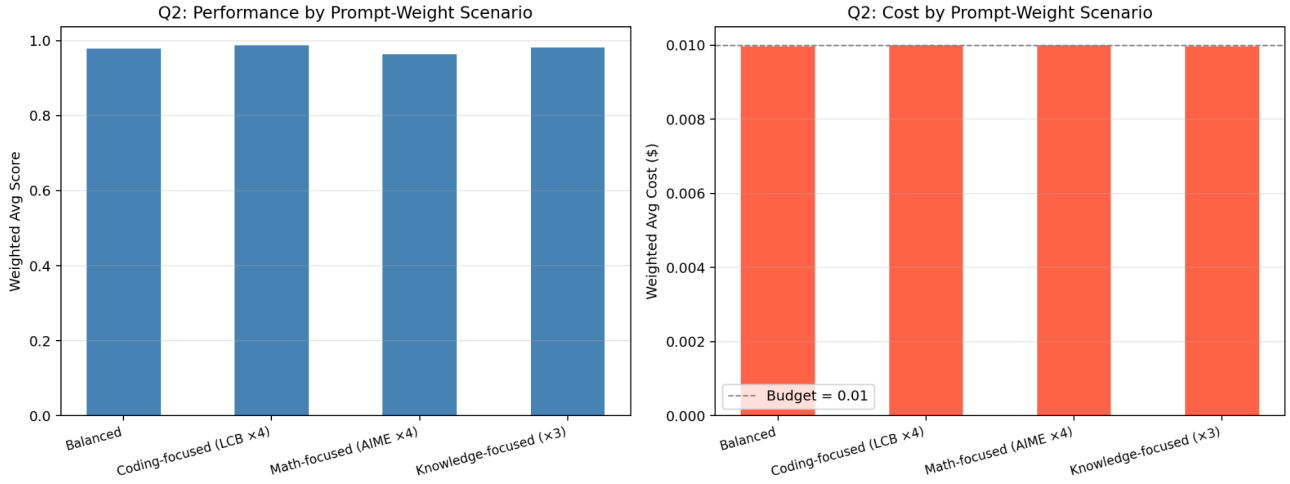

Figure A11: Performance and cost under different prompt-weight scenarios (balanced, coding-focused, math-focused, knowledge-focused). The solution remains strong across distribution shifts, with the math-focused scenario being the most challenging.

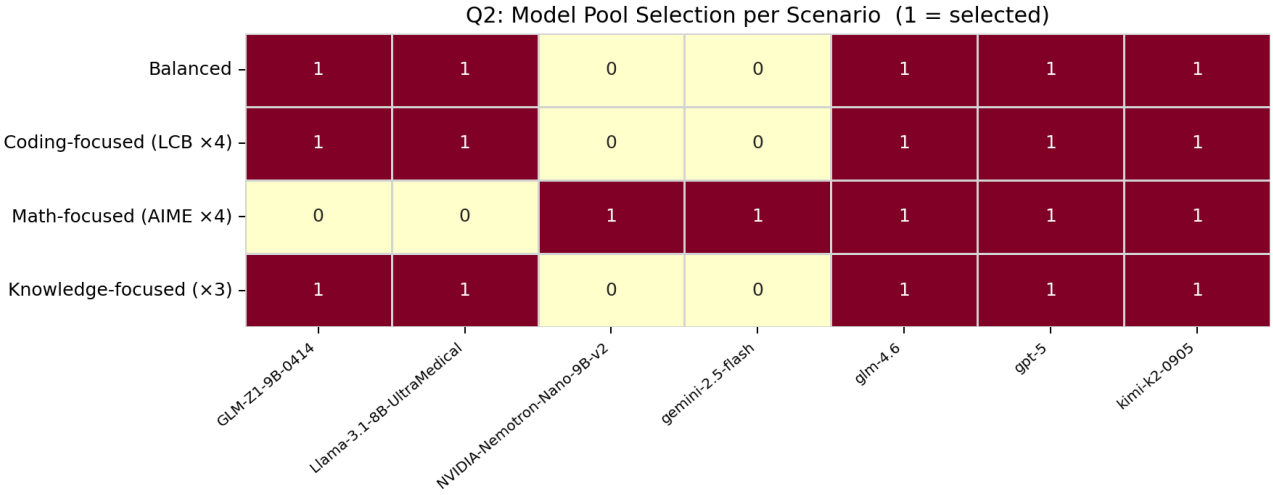

Figure A12: Selected model pools under different prompt-weight scenarios. Rows are scenarios and columns are models; 1 indicates the model is selected. Several models appear across multiple scenarios, suggesting a stable core pool.

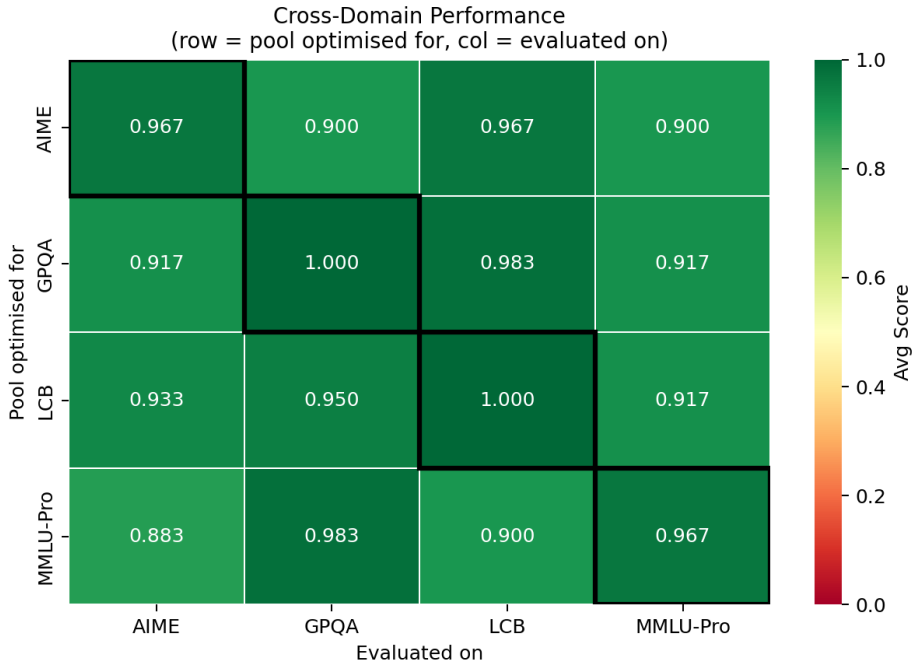

Figure A13: Cross-domain evaluation of domain-specialized pools. Diagonal entries represent in-distribution performance; off-diagonal entries measure out-of-distribution robustness.

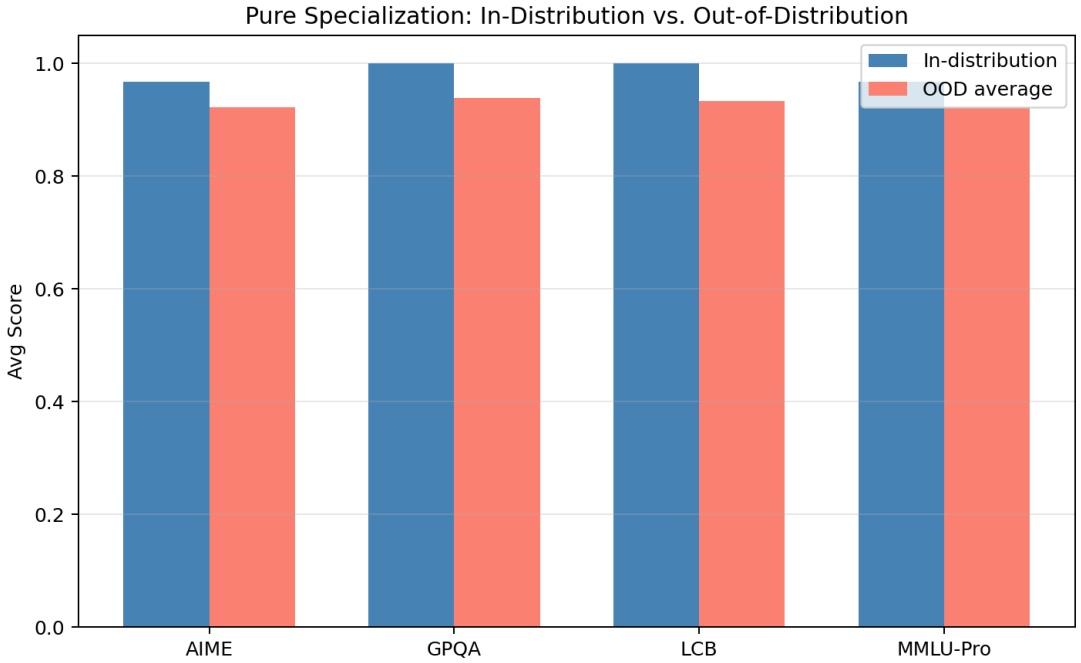

Figure A14: In-distribution versus out-of-distribution performance for specialized pools. OOD performance remains high but is consistently lower than in-distribution performance, indicating moderate sensitivity to distribution shift.

D. Pyomo Implementation

All code is organized into four modules. Below we present the core components of each file.

D.1 `util.py` — Solver Selection

```python
def choose_solver():
    candidate_solvers = ["appsi_highs"]
    for name in candidate_solvers:
        try:
            solver = pyo.SolverFactory(name)
            if solver is not None and solver.available(False):
                print(f"Using solver: {name}")
                return solver
        except Exception:
            pass
    raise RuntimeError(
        "No MILP solver found. Install HiGHS via `pip install highspy`."
    )
```

Listing 1: Solver selection utility. Tries HiGHS first; falls back to CBC or GLPK.

D.2 `data_loading.py` — Data Loading and SAA Weights

```python
def load_routerbench_data(csv_path: str) -> pd.DataFrame:
    df = pd.read_csv(csv_path)
    df["score"] = pd.to_numeric(df["score"], errors="coerce").fillna(0.0)
    df["cost"]  = pd.to_numeric(df["cost"],  errors="coerce").fillna(0.0)
    df = df.drop_duplicates(subset=["prompt_id", "model"], keep="first")
    return df

def build_prompt_weights(df: pd.DataFrame) -> dict:
    """Uniform SAA weights: w_p = 1 / |P| for all p."""
    prompts = df["prompt_id"].unique()
    return {p: 1.0 / len(prompts) for p in prompts}

def make_parameter_dicts(df: pd.DataFrame):
    """Return score[(p,m)], cost[(p,m)], and index sets."""
    score, cost, dataset_of_prompt = {}, {}, {}
    for _, row in df.iterrows():
        p, m = row["prompt_id"], row["model"]
        score[(p, m)] = float(row["score"])
        cost[(p, m)]  = float(row["cost"])
        dataset_of_prompt[p] = row["dataset"]
    prompts  = sorted(df["prompt_id"].unique())
    models   = sorted(df["model"].unique())
    datasets = sorted(df["dataset"].unique())
    return prompts, models, datasets, score, cost, dataset_of_prompt
```

Listing 2: Load RouterBench CSV and construct per-prompt SAA weights w_p.

D.3 `optimization_model.py` — Two-Stage Stochastic Model

```python
def build_and_solve(df, K, B, Q_d=None, lam=10.0,
                    dataset_multipliers=None):
    prompts, models, datasets, score, cost, d_of_p = \
```

```
        make_parameter_dicts(df)
    w = build_weighted_prompt_weights(df, dataset_multipliers)

    mdl = pyo.ConcreteModel()
    mdl.P = pyo.Set(initialize=prompts)
    mdl.M = pyo.Set(initialize=models)
    mdl.D = pyo.Set(initialize=datasets)

    # Stage-1 variable: select model pool
    mdl.y = pyo.Var(mdl.M, within=pyo.Binary)

    # Stage-2 variable: route prompt to model
    mdl.x = pyo.Var(mdl.P, mdl.M, within=pyo.Binary)

    # Auxiliary: quality violation slack per dataset
    mdl.slack = pyo.Var(mdl.D, within=pyo.NonNegativeReals)

    # Objective: maximise weighted score minus slack penalty
    mdl.obj = pyo.Objective(
        expr=sum(w[p] * score[p,m] * mdl.x[p,m]
                 for p in prompts for m in models)
             - lam * sum(mdl.slack[d] for d in datasets),
        sense=pyo.maximize
    )

    # (C1) Pool size constraint
    mdl.pool_size = pyo.Constraint(
        expr=sum(mdl.y[m] for m in models) <= K
    )

    # (C2) Each prompt assigned to exactly one model
    mdl.assign = pyo.Constraint(mdl.P, rule=lambda mdl, p:
        sum(mdl.x[p,m] for m in models) == 1
    )

    # (C3) Only route to selected models
    mdl.link = pyo.Constraint(mdl.P, mdl.M, rule=lambda mdl, p, m:
        mdl.x[p,m] <= mdl.y[m]
    )

    # (C4) Budget constraint
    mdl.budget = pyo.Constraint(
        expr=sum(w[p] * cost[p,m] * mdl.x[p,m]
                 for p in prompts for m in models) <= B
    )

    # (C5) Soft domain quality guarantee (inactive when Q_d=0)
    if Q_d is not None:
        def quality_rule(mdl, d):
            prompts_d = [p for p in prompts if d_of_p[p] == d]
            n_d = len(prompts_d)
            return (sum(score[p,m] * mdl.x[p,m]
                        for p in prompts_d for m in models) / n_d
                    >= Q_d.get(d, 0.0) - mdl.slack[d])
        mdl.quality = pyo.Constraint(mdl.D, rule=quality_rule)

    solver = choose_solver()
    solver.solve(mdl, tee=False)

    selected = [m for m in models if pyo.value(mdl.y[m]) > 0.5]
    routing  = {p: next(m for m in models
```

```
                        if pyo.value(mdl.x[p,m]) > 0.5)
                for p in prompts}
    avg_score = sum(w[p] * score[p, routing[p]] for p in prompts)
    avg_cost  = sum(w[p] * cost[p,  routing[p]] for p in prompts)

    return {"selected_models": selected,
            "routing_policy":  routing,
            "avg_score":       avg_score,
            "avg_cost":        avg_cost}
```

Listing 3: Core two-stage MILP. Stage 1 selects pool y_m; Stage 2 routes prompts x_{pm}.

D.4 `baseline_model.py` — Baseline Policies

```
def solve_weighted_sum_router(df, alpha, prompt_weights):
    """Oracle routing: unrestricted model selection per prompt."""
    # minimize avg_cost - alpha * avg_score (no pool constraint)
    ...

def evaluate_single_best(df, alpha, prompt_weights):
    """Single Best: one fixed model for all prompts."""
    best = min(models, key=lambda m:
               sum(w[p]*cost[p,m] for p in prompts)
               - alpha * sum(w[p]*score[p,m] for p in prompts))
    ...

def evaluate_single_best_per_benchmark(df, alpha, prompt_weights):
    """Single Best per Benchmark: one model per dataset."""
    for d in datasets:
        best_d = min(models, key=lambda m: objective(d, m))
        ...
```

Listing 4: Three baseline routing policies used for comparison.

D.5 `Model_Evaluation.ipynb` — Experiments

Setup

```
import sys, os
sys.path.append(os.path.abspath('..'))
from src import *

df = load_routerbench_data("../data/routerbench.csv")
```

Listing 5: Imports and data loading.

RQ1: Pool Size Sweep

```
K_VALUES, BUDGET, SLACK_PENALTY = list(range(1, 11)), 0.01, 10.0
Q_MIN = {}   # quality constraints disabled

q1_df = sweep_pool_size(
    df,
    prompt_weights   = build_weighted_prompt_weights(df),
    K_values         = K_VALUES,
    B                = BUDGET,
```

```
    Q_min_by_dataset = Q_MIN,
    slack_penalty    = SLACK_PENALTY,
)
```

Listing 6: Sweep $K = 1, \ldots, 10$ under fixed budget $B = 0.01$.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(q1_df["K"], q1_df["avg_score"],
             marker="o", linewidth=2, color="steelblue")
axes[0].set_xlabel("Pool Size K")
axes[0].set_ylabel("Weighted Avg Score")
axes[0].set_title("Q1: Performance vs. Pool Size")

for i in range(1, len(q1_df)):
    gain = q1_df["avg_score"].iloc[i] - q1_df["avg_score"].iloc[i-1]
    axes[0].annotate(f"+{gain:.3f}",
        (q1_df["K"].iloc[i], q1_df["avg_score"].iloc[i]),
        textcoords="offset points", xytext=(0, 8), fontsize=8)

axes[1].plot(q1_df["K"], q1_df["avg_cost"],
             marker="s", linewidth=2, color="tomato")
axes[1].axhline(BUDGET, linestyle="--", color="gray",
                label=f"Budget B = {BUDGET}")
axes[1].set_xlabel("Pool Size K")
axes[1].set_ylabel("Weighted Avg Cost ($)")
axes[1].set_title("Q1: Cost vs. Pool Size")
axes[1].legend()

plt.tight_layout()
plt.savefig("../output/q1_pool_size_sweep.png", dpi=180)
```

Listing 7: Plot performance and cost curves against pool size K, annotated with marginal gains.

RQ2: Prompt Distribution Scenarios

```
WEIGHT_SCENARIOS = {
    "Balanced":                {},
    "Coding-focused (LCB x4)": {"LCB":  4.0},
    "Math-focused (AIME x4)":  {"AIME": 4.0},
    "Knowledge-focused (x3)":  {"GPQA": 3.0, "MMLU-Pro": 3.0},
}

q2_df = sweep_prompt_weights(
    df,
    weight_scenarios = WEIGHT_SCENARIOS,
    K=5, B=0.01, slack_penalty=10.0,
)
```

Listing 8: Four dataset-weight scenarios representing different company focus areas.

RQ2 Extension: Pure Specialization and Cross-Domain Robustness

```
PURE_SCENARIOS = {
    "AIME only":     {"AIME": 1, "GPQA": 0, "LCB": 0, "MMLU-Pro": 0},
    "LCB only":      {"LCB":  1, "AIME": 0, "GPQA": 0, "MMLU-Pro": 0},
    "GPQA only":     {"GPQA": 1, "AIME": 0, "LCB": 0,  "MMLU-Pro": 0},
```

```
    "MMLU-Pro only": {"MMLU-Pro": 1, "AIME": 0, "GPQA": 0, "LCB": 0},
}

pure_df = sweep_prompt_weights(df, PURE_SCENARIOS, K=5, B=0.01,
                               slack_penalty=10.0)

pure_pools  = {row["scenario"].replace(" only", ""): row["selected_models"]
               for _, row in pure_df.iterrows()}
cross_mat   = cross_domain_matrix(df, pure_pools)

# In-distribution vs OOD comparison
datasets_order = cross_mat.columns.tolist()
in_dist  = [cross_mat.loc[d, d] for d in datasets_order]
ood_mean = [cross_mat.loc[d, [c for c in datasets_order if c != d]].mean()
            for d in datasets_order]
```

Listing 9: Optimize one pool per dataset; evaluate cross-domain performance matrix.